# DD2482 Project Report

Amin Nouiser, Ivar Boqvist

October 2026

## 1 Introduction

In this project, we've built a DevOps pipeline around a Pomodoro timer web app. The application consists of three main parts: a backend written in Python which uses the FastAPI web framework, a frontend written in TypeScript which uses the Svelte UI framework, and a relational database in the form of PostgreSQL.

The code[1] is hosted on GitHub, where new contributions are made using pull requests, or PRs. GitHub Actions are used to lint, type-check, test, build, and package the apps as container images on GitHub Container Registry (GHCR). Terraform is used to provision an infrastructure on AWS where the containers are deployed.

## 2 The DevOps pipe

An illustration of the pipeline can be seen in *Figure 1*.

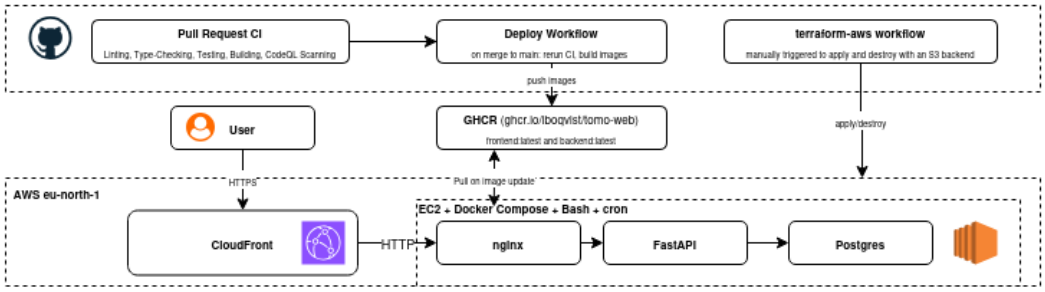



Figure 1: The Pipeline architecture

### 2.1 CI

We're using GitHub Actions for our CI pipeline. There are three custom-defined CI workflows: one for the backend (including the database), one for the frontend, and one for the infrastructure. Each workflow is defined as a YAML file using GitHub's workflow syntax under .github/workflows. These workflows run automatically on each pull request, but can be called upon manually if needed. To avoid having to wait for workflows to run when no changes have been made, each layer's workflow triggers only on changes made to relevant folders or files.

The backend CI workflow installs dependencies, checks for linting errors, formatting errors, type checking errors, and runs tests on the Python backend. We're using uv as our package and project

[1] https://github.com/iboqvist/tomo-web

manager, which replaces the roles of both both pip and poetry in a traditional Python stack. Dependencies are defined in the pyproject.toml file alongside versioning constraints and tool configurations. uv uses the dependencies and constraints in this file to generate a lockfile, which is then used in the CI workflow to pin exact versions of the production dependencies.

As for our other development dependencies: we use ruff as a linter and formatter, pyrefly as a type checker and language server, and pytest as a unit testing suite. ruff was chosen as it's fast, configurable, and supports both linting and formatting. Integrating linting and formatting in both the local development environment and CI pipeline helps the code stay consistent, idiomatic, and legible. We also use pyrefly for type checking. Python is dynamically typed, but with good annotation praxis and a type checker it becomes harder to shoot yourself in the foot. As for pytest, we chose it as it's well-documented and easy to use. Each step is called sequentially, so an early failure will short-circuit the workflow checks as to not waste time or compute.

The frontend CI workflow has a similar structure. The workflow starts by installing dependencies, then checks for linting errors, formatting errors, type errors, and finally builds the application. Unlike Python, TypeScript is a transpiled language, so the last step is required. We're using Bun as our TypeScript runtime, package manager, and bundler. oxlint and oxfmt are used for linting and formatting respectively. Unlike in Python, the configuration files for oxlint and oxfmt are separate from the dependency definitions. We then run Bun's built-in type checker. After all checks have passed, Bun is then used to build the backend, transpiling the TypeScript code into JavaScript that can then be run by the underlying JS engine.

The infrastructure CI workflow checks the formatting of and the syntax validity of the Terraform configuration. It uses the Terraform CLI for this, and so doesn't require any third-party, non-transient dependencies aside from Terraform itself.

## 2.2 Security automation

We have Dependabot and CodeQL set up on our GitHub repo for security automation purposes.

Dependabot is a dependency management automation used to monitor and scan for package updates and security patches. If a security vulnerability is found in a dependency, transient or not, Dependabot will post an alert on your repo. For projects where dependencies are defined declaratively in a package- or lockfile, such as this project's uv and Bun setups, it will also open PRs to bump vulnerable dependencies to a safe version.

CodeQL is a semantic code analysis engine and toolchain for code analysis. There's a GitHub integration that allows for querying code using CodeQL and thereby identifying potential vulnerabilities or insecure patterns. Unlike Dependabot, CodeQL's security checks don't run continuously, but rather along with our other workflows when code is pushed to a PR. These checks don't block PRs from being merged, but rather serve to highlight potential security improvements.

## 2.3 CD

Deployment happens in two stages, namely delivering the container images and setting the infrastructure that runs them.

The first stage happens in the *cd-deploy* workflow which is triggered on pushes to main that changes the frontend or backend. It calls the integration workflows as reusable workflows. If they pass, frontend and backend images are built and published with the *:latest* tags on GHCR.

The second stage is the *terraform-aws* workflow which creates or tears down the AWS infrastructure. Unlike the other workflows, it is not triggered automatically but only manually through a *workflow_dispatch* trigger where we choose *apply* or *destroy*. We made it manual so that changes to Terraform code would not start a runner that takes 10 minutes to complete and may not be needed at the time. The infrastructure wasn't needed to be up at all times from a user perspective either. A real production pipeline could add a push trigger instead, ideally with a terraform plan step that requires reviewing before anything is applied. A *concurrency group* prevents two runs from overlapping, which

could otherwise lead to an inconsistent backend file. The workflow authenticates to AWS using access keys of an IAM user stored as GitHub secrets.

The last step happens in the EC2 instance itself. A cron job, scheduled every two minutes, runs a bash script that pulls the frontend and backend images and restarts the containers if the image has updated. We chose this instead of a push approach from GitHub as that would require access to the instance and may induce a threat on the system.

### 2.4 Infrastructure

The infrastructures main components is an EC2 instance and CloudFront on AWS. All resources are defined with Terraform, using community modules and resources by the AWS provider from the Terraform registry. The AWS provider version is pinned and the *.terraform.lock.hcl* file is committed so that every run uses the same version.

The Terraform backend is stored in an S3 bucket, so it persists between workflow runs and is shared between developer machines and GitHub actions. `use_lockfile = true` prevents concurrent changes on the backend file, and in turn the infrastructure as well. Since terraform can't create the backend which it depends on, the S3 bucket has to be created manually before the first `terraform init`.

The app is deployed using a Docker Compose file and a bash script which are passed to the instance as *user data* through a Terraform template file. On creation, the script installs necessary components and sets the bash script as described in section 2.3. `user_data_replace_on_change` is enabled on the instance so that any changes to the script or compose file makes terraform replace the instance on `terraform apply`.

CloudFront is the only public entrypoint to the App. It serves a certificate and gives our deployment a subdomain under `.cloudfront.net` so that our visitors can reach the app over HTTPS. Traffic between CloudFront and the instance happens over HTTP. AWS Session manager is used to manage the ec2 instance over a secure channel.

## 3 Limitations

Our setup works well for a demo, but it has some limitations and points of potential improvement that should likely be addressed before going into a theoretical production environment.

The application runs on a single EC2 instance in a single availability zone, meaning that if either of them fails, so does the app. A production setup would instead run multiple, redundant instances across different availability zones. Alternatively, you could rewrite the deployment as to work in a Kubernetes cluster.

Apart from the deploy script writing to syslog, we have no monitoring in our setup which means that we are limited in our ability to detect problems in our infrastructure. This could be improved by provisioning a log or metric observability platform like Splunk or Prometheus.

One thing that we could have added is a staging environment. Currently, changes go straight from CI to production, so there is no separate infrastructure to test it on before it reaches the users. An improvement could deploy a second copy of the infrastructure with Terraform and give each environment a separate container image tag. The staging environment could then run the *:staging* image before it is promoted to *:production* after review.

## 4 AI usage

The web application that our pipeline is built around was written nearly entirely by an LLM. Some modifications have been done by hand, such as swapping out the previous DynamoDB database for PostgreSQL. No part of the DevOps infrastructure as presented in this project has been written by or with the help of an LLM or any other kind of AI.